PERFORMANCE ENGINEERING · REFERENCE DOCUMENT

Load Testing Mathematics & Engineering

From virtual users to server capacity — what actually happens between the moment you configure a load test and the moment the final graph appears on your screen.

CENTRAL QUESTION	“I want to test 500 users at 20 requests/second” — what really happens?
WORKED EXAMPLE	Pocket Coach — 5,000 registered users, 500 concurrent, PostgreSQL + Redis
SCOPE	Tester configuration → tool → operating system → network → server → database → metrics
MATHEMATICS	Every figure computed and cross-checked for internal consistency
AUDIENCE	SQA engineers, performance engineers, and the people who read their reports

Contents

PART V · REFERENCE

PART I

The chain

1 The story: what your manager asked for

You are the load tester on **Pocket Coach**, a coaching application. Your manager says:

> **THE BRIEF**
>
> “We expect 5,000 registered users, with about 500 of them online at peak. Find out whether our system can handle them.”

This sentence contains no testable instruction. “500 users” is not a load — it is a population. Before anything can be measured, it has to be converted into quantities a machine can generate. That conversion is the first real skill in performance testing, and most bad load tests fail here rather than in the tool.

The system under test

COMPONENT	SPECIFICATION	WHY IT MATTERS TO US
Load balancer	Single entry point	Connection limits, queueing, timeouts
Application server	16 CPU cores, 32 GB RAM	CPU is usually the first ceiling
PostgreSQL	Connection pool, disk I/O	Pool exhaustion appears as latency, not errors
Redis cache	In front of read queries	Hit ratio decides how much load reaches the database
Load generator	Your laptop or a CI runner	Can silently become the bottleneck

What the tester actually decides

A tester does not “create 500 requests”. They choose a set of parameters, and the tool derives the traffic from them:

TEST CONFIGURATION — THE TESTER'S INPUTS

```
Virtual users (VUs)     how many simulated people exist at once
Arrival rate / RPS      how many requests should start per second
Duration                how long the test runs
Ramp-up                 how quickly load is introduced
Iterations              how many times each user repeats the journey
Scenario                which requests, in what order
Think time              how long a user pauses between actions
Thresholds              what counts as pass or fail
```

Every number in the rest of this document flows from those eight inputs.

2 The full chain, stage by stage

Here is what happens between pressing *Start* and reading a graph. Each stage transforms the load, and each one can distort the result.

`TESTER`	Chooses VUs, rate, duration, ramp, think time and thresholds. **Decides what "500 users" means numerically.**
`TOOL`	Converts configuration into a schedule: when each virtual user starts, what it executes, when it repeats. Allocates workers and metric buffers.
`WORKERS`	Each VU runs the scenario — build request, serialise body, apply auth, wait for response, record timing, sleep for think time, repeat.
`OS`	Allocates sockets and file descriptors, schedules threads onto cores, manages TCP state, performs TLS encryption. **Finite resources here cap your achievable load.**
`NETWORK`	Bytes travel. Adds propagation delay, bandwidth limits, packet loss, and possible queueing at every hop.
`BALANCER`	Terminates or forwards connections, spreads requests across backends, applies its own connection limits and timeouts.
`APP SERVER`	Accepts the connection, may **queue** the request until a worker is free, parses, authenticates, executes business logic.
`DATA TIER`	Cache lookup; on a miss, a database connection is borrowed from the pool, queries run, locks may be taken, results return.
`RESPONSE`	Serialised and sent back down the same path, arriving as bytes at the generator.
`TOOL`	Stops the clock, checks status and assertions, tags the sample, and adds it to in-memory aggregates.
`METRICS`	Counts, rates, averages and percentiles are computed — often from sampled data, not every observation.
`ANALYSIS`	You decide whether the system passed. **This is the only stage that produces a decision, and it inherits every distortion above it.**

> `THE MEASUREMENT PRINCIPLE`
>
> The tool measures **elapsed time at the client** — from just before the request is written to just after the response is read. That figure necessarily includes network travel, server queueing, and a share of the generator's own scheduling. It is *not* your server's processing time, and treating it as such is the most common analytical error in load testing.

3 Requests, responses, transactions, iterations

These four words are used loosely in conversation and precisely in tooling. Mixing them up produces reports that disagree with themselves.

TERM	DEFINITION	EVERYDAY ANALOGY
Request	One HTTP message sent to the server	One question asked
Response	One HTTP message returned, carrying a status code	One answer received
Transaction	A business action, which may span several requests	“Place an order”
Iteration	One full pass of a virtual user through the scenario	One lap of the shop

The counting arithmetic

Our Pocket Coach journey has four steps:

ONE ITERATION

```
Login  →  Get Profile  →  Get Feed  →  Get Posts

1 iteration = 4 requests
```

SCALING IT UP

```
Users                  = 100
Iterations per user    = 10

Total iterations       = 100 × 10
                       = 1,000

Total requests         = 1,000 × 4
                       = 4,000 requests
```

EXACT

$$\text{Requests} = \text{Users} \times \text{Iterations per user} \times \text{Requests per iteration}$$

Exact when every iteration completes and no request is retried or redirected. Both of those assumptions break in practice — see §32 for retries and the note below for redirects.

REDIRECTS INFLATE THE COUNT

If a request returns `302` and the tool follows it, that single logical step costs **two** HTTP requests. A four-step journey where three steps redirect actually issues seven requests, not four. Reports that show “requests” far above *iterations* × *steps* are usually explained by redirects, not by a bug.

4 Virtual users and the four kinds of “user”

The word “user” carries four different meanings in a capacity conversation. Confusing them is how a test ends up 50× too aggressive or 50× too gentle.

TERM	MEANING	POCKET COACH
Registered users	Rows in your users table	`5,000`
Active users	Used the app in a period (daily actives)	`~1,500/day`
Concurrent users	Have a live session at the same instant	`500`
Virtual users (VUs)	Simulated workers in the load tool	`a chosen number`

THE INEQUALITY THAT MATTERS

`5,000 registered ≠ 500 concurrent ≠ 50 requests/second ≠ 500 VUs`

These are four different quantities with four different units. Only the last two can be typed into a load-testing tool, and only the third describes actual pressure on the server.

What a virtual user actually is

A VU is **a worker that performs one action at a time and then repeats**. It is not a request, and it is not a person. Crucially, a VU spends most of its life *waiting* — for a response, or for its think time to elapse.

A USER'S DUTY CYCLE — POCKET COACH

```
Requests per user per minute  = 6
Time between requests         = 60 / 6      = 10 s
Response time                 = 0.2 s

Fraction of time actually in the system:
                              = 0.2 / 10    = 2%
```

Each user is "in" the system only **2% of the time**. That single number is why 500 concurrent users do not mean 500 simultaneous requests — a point §5 makes exact.

TOOL CONCEPT	WHAT IT MAPS TO	CONSEQUENCE
Thread	An OS thread, one per VU in thread-based tools	Memory and scheduling cost per VU
Worker / goroutine	Lightweight task multiplexed onto few threads	Many more VUs per machine
Process	Separate OS process, often one per CPU core	Isolation, but higher memory

Different tools choose different models. §17 and §28 cover why this changes what one machine can generate.

5 Concurrency and Little's Law

This is the single most useful equation in performance engineering. It relates how many things are in a system, how fast they arrive, and how long they stay.

EXACT

$$L = \lambda \times W$$

L — average number of items in the system · **λ** — average arrival rate · **W** — average time each item spends in the system.
Little's Law is a *theorem*, not an approximation: it holds for any stable system regardless of arrival distribution or service order. The conditions are that the system is observed over a long period and is not accumulating work indefinitely.

Applied to requests

HOW MANY REQUESTS ARE IN FLIGHT AT ONCE?

```
λ (throughput)     = 50 requests/second
W (response time)  = 0.2 seconds

L = 50 × 0.2
  = 10 requests in flight at any instant
```

THE RESULT THAT SURPRISES PEOPLE

Pocket Coach has **500 concurrent users**, but only **10 requests** are being processed at any given moment. The other 490 users are reading, typing, or idle. Sizing your server for 500 simultaneous requests would over-provision it by 50×.

Applied to users — the closed-model form

When a fixed population of users loops through a journey, each user's cycle is *response time plus think time*. Little's Law then reads:

EXACT

$$N = \lambda \times (R + Z)$$

N — number of users · **R** — response time · **Z** — think time. This is Little's Law with the “system” drawn around the users rather than the requests.

CHECKING POCKET COACH FOR CONSISTENCY

```
N = 500 users
R = 0.2 s
Z = 9.8 s      (10 s between requests, minus 0.2 s of response)

λ = N / (R + Z)
  = 500 / (0.2 + 9.8)
  = 500 / 10
  = 50 requests/second      ✓ matches the 6 req/user/min figure
```

Two independent routes — counting requests per user per minute, and applying Little's Law — produce the same 50 RPS. That agreement is what tells you the model is sound.

Why a slower server needs more capacity at the same RPS

RESPONSE TIME DEGRADES, CONCURRENCY RISES

```
Healthy:   L = 20 × 0.5  = 10 concurrent requests
Degraded:  L = 20 × 2.0  = 40 concurrent requests
```

Throughput is unchanged at 20 RPS, yet the server must now hold **four times** as many requests in memory simultaneously — four times the worker threads, connections and buffers. This is the mechanism behind most cascading failures: a small latency increase multiplies concurrency, concurrency exhausts a pool, and exhaustion multiplies latency again.

ASSUMPTIONS TO STATE OUT LOUD

Little's Law needs a **stable** system — arrivals must not persistently exceed completions. During a spike, or once a queue is growing without bound, the system is not in steady state and `L = λW` will not describe it correctly. It is exact for the averages over a long observation window, not for any single instant.

PART II

The core numbers

6 Throughput: RPS, TPS and conversions

Throughput is work completed per unit time. It is the measure of how much your system actually did — as opposed to how many users were present.

EXACT

$$RPS = Total\ Requests \div Total\ Seconds$$

An average over the window. It says nothing about the shape of the load inside that window — a test that idles for 5 minutes then bursts will report the same average as a perfectly steady one.

BASIC CONVERSION

```
12,000 requests over 10 minutes

10 minutes  = 600 seconds
RPS         = 12,000 / 600
            = 20 requests/second
```

UNIT	CONVERSION	AT 20 RPS	TYPICALLY USED BY
Requests/second	base unit	20	Load tools, engineers
Requests/minute	× 60	1,200	APM dashboards
Requests/hour	× 3,600	72,000	Capacity planning
Requests/day	× 86,400	1,728,000	Billing, business reports

RPS vs TPS

Transactions per second counts business actions, not HTTP calls. If one checkout transaction issues 4 requests, then 20 TPS is 80 RPS. Always ask which unit a stakeholder means — a fourfold misunderstanding is easy here.

THROUGHPUT IS AN OUTPUT, NOT AN INPUT

In a closed model (fixed VUs) you cannot *set* throughput — it emerges from how fast the server responds. If the server slows down, throughput falls even though your VU count is unchanged. Only an **arrival-rate** model (§33) lets you specify RPS directly.

7 Iterations and total request count

EXACT

$$\text{Total Iterations} = \text{Users} \times \text{Iterations per user}$$

EXAMPLE A — FIXED WORK

```
50 users × 20 iterations
  = 1,000 iterations

× 4 requests per iteration
  = 4,000 requests
```

EXAMPLE B — FIXED DURATION

```
Duration          = 300 s
Iteration takes   = 6.3 s
Iterations per VU = 300 / 6.3 ≈ 47

100 VUs × 47   = 4,700 iterations
× 4 requests   = 18,800 requests
```

Example B is how most real tests behave: you fix the *duration*, and the iteration count falls out of how quickly each user can complete a lap. That is why **a faster server produces more requests in the same test** — and why comparing raw request counts between two runs can mislead.

8 Duration and arrival rate

EXACT

$$\text{Total Requests} = \text{Arrival Rate} \times \text{Duration}$$

Exact in an *open* model, where the tool starts requests on a clock regardless of whether earlier ones have finished.

ARRIVAL-RATE TEST

```
20 requests/second × 300 seconds
  = 6,000 requests

This holds whether the server responds in
20 ms or 2,000 ms — the schedule is fixed.
```

Contrast with 100 VUs running continuously

CLOSED MODEL, SAME DURATION

```
100 VUs, R = 0.2 s, Z = 1.8 s

λ = N / (R + Z) = 100 / 2.0   = 50 RPS
Total = 50 × 300              = 15,000 requests

Now the server slows to R = 1.8 s:
λ = 100 / (1.8 + 1.8)         = 27.8 RPS
Total = 27.8 × 300            ≈ 8,340 requests
```

THE CRITICAL DIFFERENCE

When the server degrades, the **closed** model automatically sends less traffic — users are stuck waiting, so they stop asking for more. The **open** model keeps sending at the configured rate and lets the queue grow. Real customers behave like the open model: they arrive when they arrive, whether or not you are ready.

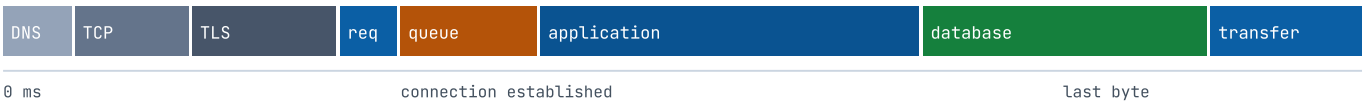
9 Response time and its components

EXACT

$$\text{Response Time} = t_{\text{last byte received}} - t_{\text{first byte sent}}$$

As observed by the client. This is what a load-testing tool reports by default.

ANATOMY OF A 200 ms RESPONSE (client-observed)



First request on a new connection pays DNS + TCP + TLS.
With keep-alive, subsequent requests skip all three and start at "req".
Queue time is invisible to the server's own logs — it is counted before your code runs.

COMPONENT ARITHMETIC (WARM CONNECTION)

```
Network round trip        =  40 ms
Server queue wait         =  20 ms
Application processing    =  80 ms
Database processing       =  40 ms
Response transfer         =  20 ms
                             ─────
Client-observed total     = 200 ms
```

WHY YOUR APM AND YOUR LOAD TEST DISAGREE

The application's own timer starts when your handler is entered. It would report **120 ms** (application + database) for the request above. The load tester sees **200 ms**. Neither is wrong — they measure different spans. The 80 ms difference is network and queueing, and *your users experience it*. Optimising only what the APM shows can leave the actual complaint untouched.

10 Latency vs response time

“Latency” is used to mean at least three different things. Define which one you mean in every report.

TERM	MEASURES	OUR EXAMPLE
Network latency (RTT)	Round trip for a packet, no processing	40 ms
Queue time	Waiting for a free worker before processing	20 ms
Server processing time	Application + database work	120 ms
Time to first byte (TTFB)	Until the first response byte arrives	180 ms
Response time	Until the *last* byte arrives	200 ms

TTFB excludes the body transfer; response time includes it. For large payloads the gap is significant.

WHY BACKEND-ONLY MEASUREMENT MISLEADS

A team reporting “our p95 is 120 ms” from application metrics, while users experience 200 ms, is not lying — it is answering a different question. Under load the gap *widens*, because queue time grows fastest of all the components. A backend-only view can look flat while the customer-facing number doubles.

11 Percentiles and why the average lies

Take 100 requests. Ninety-five are healthy at 140–160 ms. Five hit a slow path at 2.8–3.2 s.

THE SAME DATASET, SIX DIFFERENT SUMMARIES

```
Dataset:  95 requests at 140–160 ms
           5 requests at 2,800–3,200 ms

Mean (average)      =   291.9 ms
Median (p50)        =   150 ms
p90                 =   159 ms
p95                 =   292 ms
p99                 = 3,101 ms
Maximum             = 3,200 ms
```

THE AVERAGE DESCRIBES NOBODY

The mean is **291.9 ms** — yet **only 5 of the 100 requests** were slower than that. Ninety-five users had an experience roughly *twice as fast* as the "average", and five had one *ten times slower*. No user experienced 292 ms. The average is a number that describes the arithmetic, not the customer.

How to read each percentile

STATISTIC	MEANING	USE IT FOR
p50 / median	Half of requests were faster	The typical experience
p90	9 in 10 were faster	Everyday quality
p95	19 in 20 were faster	**The usual SLA target**
p99	99 in 100 were faster	Tail pain; where outages hide
p99.9	999 in 1,000 were faster	High-volume systems only
Maximum	The single worst sample	Little value — one blip sets it

PERCENTILES DO NOT AVERAGE

You **cannot** average two p95 values to get a combined p95, and you cannot average p95 across time buckets. Doing so is a common and serious reporting error. Percentiles must be recomputed from the underlying samples — which is why tools keep either all samples or a statistical sketch of them.

A typical target pair is `p95 < 500 ms` and `p99 < 1 s`. This says "almost everyone is fast, and even the unlucky are not abandoned" — a claim an average can never make.

12 Error rate and HTTP status meanings

EXACT

$$\text{Error Rate} = \text{Failed} \div \text{Total} \times 100$$

WORKED EXAMPLE

```
Total requests   = 10,000
Failed requests  =    500

Error rate    = 500 / 10,000 × 100   = 5.0%
Success rate  = 9,500 / 10,000 × 100 = 95.0%
```

CODE	MEANING	IN A LOAD TEST IT USUALLY INDICATES
400	Bad Request	Your test data or payload is wrong — a *test* defect
401	Unauthorized	Token missing or expired mid-run
403	Forbidden	Authenticated but not permitted — often wrong role
404	Not Found	Placeholder IDs that were never filled in
409	Conflict	Concurrency collision — a genuine finding
422	Unprocessable	Valid JSON, invalid business content
429	Too Many Requests	Rate limiting engaged — you found a real ceiling
500	Internal Server Error	Unhandled exception — the most serious signal
502	Bad Gateway	Backend died or returned garbage to the proxy
503	Service Unavailable	Overloaded or shedding load deliberately
504	Gateway Timeout	Backend too slow — classic saturation symptom

TRIAGE RULE

4xx errors usually mean your test is wrong. 5xx errors usually mean the system is wrong. A wall of 401s or 404s at the start of a run almost always means missing tokens or unfilled ID placeholders — fix the test before reporting an incident. A rising rate of 500/503/504 *as load increases* is the real thing you came to find.

13 Ramp-up arithmetic

0 → 100 VUS OVER 5 MINUTES

```
Users to add   = 100
Ramp duration  = 5 minutes = 300 seconds

Arrival rate of users = 100 / 300
                      = 0.333 users/second
                      ≈ one new user every 3 seconds
```

Ramp-up exists for three reasons, and skipping it invalidates results in three ways:

- **Caches warm up.** The first requests hit cold caches and empty connection pools. Starting at full load measures your cold-start path, not your steady state.
- **Autoscaling needs time.** If your platform adds instances on demand, an instant jump measures the scaling lag, not the scaled system.
- **You can see *where* it breaks.** A gradual ramp shows the load level at which latency turns upward. A step function tells you only pass or fail.

RAMP TIME IS NOT TEST TIME

Averages computed over the whole run include the lightly loaded ramp period and will flatter your results. Report the **steady-state window** separately — typically from the end of the ramp to the start of the ramp-down.

14 Spike testing

50 → 1,000 VUS IN 10 SECONDS

```
Increase       = 1,000 - 50 = 950 VUs
Over           = 10 seconds

Rate of arrival = 95 new virtual users per second
Multiple        = 20× the starting load
```

What each layer experiences in those ten seconds:

LAYER	WHAT HAPPENS	FAILURE MODE
Load generator	950 workers created, sockets opened	Generator saturates first and never delivers the spike
Network / OS	Burst of TCP handshakes, TLS negotiations	Ephemeral port or file-descriptor exhaustion
Load balancer	Connection table fills	New connections refused or queued
App server	Worker pool fully occupied	Requests queue; latency climbs before any error appears
Connection pool	All DB connections borrowed	Threads block waiting for a connection
Database	Query concurrency spikes; locks contend	Lock waits, then timeouts
CPU / RAM	Utilisation jumps; garbage collection intensifies	GC pauses add latency exactly when you can least afford it

WHAT A SPIKE TEST IS REALLY ASKING

Not “can we serve 1,000 users?” but **“do we degrade gracefully or collapse, and do we recover?”** The most valuable part of a spike test is the minute *after* the spike ends. A system that returns to baseline latency has healthy back-pressure. One that stays degraded has a queue it cannot drain, or a leak.

15 Think time

Think time is the pause between a user's actions — reading a screen, typing, deciding. It is the difference between simulating people and simulating a denial-of-service attack.

A REALISTIC JOURNEY

```
GET  /feed        200 ms   →  think 3 s
GET  /profile     150 ms   →  think 5 s
POST /comment     250 ms   →  think 4 s
                  ──────      ──────
Work              600 ms     Waiting  12 s

Iteration = 12.6 s, of which 4.8% is server work
```

Think time is a throughput dial

The same 100 VUs produce wildly different load depending only on think time:

SCENARIO	R	Z	RPS FROM 100 VUS
No think time (machine-gun)	0.2 s	0 s	500
1 second think time	0.2 s	1 s	83.3
Realistic user (Pocket Coach)	0.2 s	9.8 s	10
Slow API, no think time	2.0 s	0 s	50

All four rows use λ = N / (R + Z) with N = 100. The first and third differ by **50×** — same "100 users".

WHY TWO TEAMS BOTH TESTING "100 USERS" GET DIFFERENT ANSWERS

Because "100 users" is not a load specification. Without stating think time, a VU count communicates almost nothing. **Always report VUs and think time together, or report throughput instead.**

ZERO THINK TIME IS NOT "WORST CASE" — IT IS A DIFFERENT CASE

Removing think time is sometimes defended as conservative. It does produce more load, but it also produces *unrealistic* load: perfect connection reuse, hot caches, and no idle connections held open. It is a legitimate way to find a throughput ceiling; it is not a simulation of your users.

PART III

The machinery

16 Connections and protocol behaviour

Every request travels over a TCP connection. Whether that connection is new or reused changes both what you measure and what you cost the server.

COST OF A COLD CONNECTION VS A WARM ONE

```
New connection (HTTPS):
  TCP handshake      ~1 round trip
  TLS handshake      ~1–2 round trips
  Request            ~1 round trip
                     ————————————
                     3–4 round trips

Reused connection (keep-alive):
  Request            ~1 round trip

On a 40 ms round trip, that is roughly
160 ms versus 40 ms for the same request.
```

PROTOCOL	CONCURRENCY MODEL	EFFECT ON LOAD TESTING
HTTP/1.1	One request at a time per connection; browsers open ~6 per host	Concurrency is bounded by connection count; head-of-line blocking
HTTP/2	Many streams multiplexed over one connection	Far fewer sockets for the same load; one connection can carry huge concurrency
HTTP/3 (QUIC)	Multiplexed over UDP, no TCP head-of-line blocking	Better on lossy networks; different OS resource profile

TWO SETTINGS THAT SILENTLY CHANGE YOUR RESULTS

Connection reuse — disabling keep-alive multiplies handshakes, inflating both measured latency and generator CPU (TLS is expensive). **Redirect following** — doubles the request count for every redirecting endpoint. Both are defensible choices; both must be stated when you report numbers, because they are not comparable across runs that configured them differently.

17 Inside the load generator machine

This section is the one most often skipped, and it is where untrustworthy results are born. The generator is a computer running a demanding program, subject to exactly the same limits as the system it is testing.

RESOURCE	WHAT CONSUMES IT	SYMPTOM WHEN EXHAUSTED
CPU	TLS encryption, JSON parsing, metric aggregation, scheduling	Requests are issued late; measured latency inflates
RAM	Per-VU state, response bodies, buffered results	Swapping, or an out-of-memory crash mid-run
Sockets / ports	One ephemeral port per connection; ports linger in TIME_WAIT	Connection failures that look like server errors
File descriptors	Every socket and open file	"Too many open files" — a generator fault, not a server fault
Threads	Thread-per-VU tools allocate one per user	Context-switching overhead grows faster than useful work
Garbage collection	Managed runtimes (JVM, .NET) pause to reclaim memory	Pauses land inside measurements, inflating the tail
Disk / logging	Writing every sample to a results file	I/O stalls that appear as latency spikes
Network	Aggregate bandwidth of all VUs	Link saturation caps throughput regardless of VU count

THE LOAD GENERATOR IS A SYSTEM UNDER TEST TOO

Every millisecond the generator spends waiting for a CPU slice, running garbage collection, or blocking on a log write is a millisecond that lands in your latency figure — attributed to the server. **If you do not monitor the generator, you cannot know whether you measured the system or the measuring device.**

18 Load generator overhead

Two different overheads exist and they must never be conflated:

TARGET-SYSTEM OVERHEAD

```
Time the server genuinely
spends on the request:

  queueing
+ application logic
+ database work
+ serialisation

This is what you want to measure.
```

LOAD-GENERATOR OVERHEAD

```
Time added by the measuring
device before and after:

  scheduling delay
+ TLS work
+ request construction
+ response parsing
+ metric recording

This contaminates what you measure.
```

DO NOT ACCEPT A FIXED NUMBER FOR GENERATOR OVERHEAD

You will see claims that a given tool "adds *X* milliseconds per request". **Treat any such fixed figure as unreliable.** Generator overhead is not a constant: it depends on concurrency, payload size, TLS, connection reuse, logging, CPU headroom and the runtime's garbage collector. The *only* valid figure is one measured in your own environment, at the concurrency your real test will use — see §29 for how.

Why a saturated generator invalidates a test

A RESULT YOU MUST DISCARD

```
Target API reports  :  p95 = 500 ms
Load generator      :  CPU = 95%, network saturated

Two problems, both fatal:

1. The generator cannot issue requests on schedule,
   so the target never received the intended load.

2. Time spent queueing inside the generator is
   counted as response time, inflating every figure.
```

THE RULE

If the load generator cannot generate the requested load, the target server never received the intended load — and any conclusion about the target's capacity is unfounded. A test where the generator is pinned tells you about the generator.

19 Distributed load generation

When one machine cannot produce the required load, you run several — a controller coordinating multiple workers, with results merged afterwards.

NAIVE DIVISION

```
Target load           = 10,000 RPS
Measured per machine  =  2,000 RPS

Generators needed = 10,000 / 2,000
                  = 5
```

FIVE IS THE WRONG ANSWER

That calculation runs every machine at **100% of its measured ceiling** — precisely the saturated condition §18 says invalidates results. Size for headroom instead:

SIZING WITH HEADROOM

```
Usable per machine at 70% = 2,000 × 0.70 = 1,400 RPS

Generators needed = 10,000 / 1,400
                  = 7.14  →  8 machines
```

CONSIDERATION	WHY IT COMPLICATES DISTRIBUTED TESTING
Result merging	Percentiles cannot be averaged across machines (§11); raw samples or sketches must be combined
Clock skew	Machines disagree on time, blurring the timeline
Uneven capacity	A slower machine silently contributes less load than assumed
Network position	Generators in different regions measure different latencies
Coordination cost	The controller itself consumes resources and can bottleneck

20 CPU calculations and the Utilization Law

EXACT

$$U = X \times D$$

U — utilisation (in CPU-seconds per second, i.e. cores busy) · **X** — throughput · **D** — service demand, the CPU-seconds one request consumes. This is the Utilization Law from queueing theory; it is exact for a given resource. **D must be measured**, not guessed.

POCKET COACH CPU MODEL

```
Measured service demand  D = 0.040 s CPU per request
Throughput               X = 50 requests/second
Cores available            = 16

Cores busy   = 50 × 0.040     = 2.0 cores
Utilisation  = 2.0 / 16 × 100 = 12.5%
```

WHERE THE CPU CEILING IS

```
Theoretical maximum  = cores / D
                     = 16 / 0.040
                     = 400 requests/second

Safe target at 70%   = (16 × 0.70) / 0.040
                     = 280 requests/second
```

LIMITS OF THIS SIMPLIFICATION

- **Perfect parallelism is assumed.** Real workloads have serial sections; Amdahl's law caps the benefit of extra cores.
- **Hyper-threaded "cores" are not full cores.** 16 logical threads on 8 physical cores deliver well under 16 cores of work.
- **Utilisation is not linear near saturation.** Past roughly 70–80%, queueing makes response time rise sharply — see §24.
- **D is not constant.** Cache misses, garbage collection and larger payloads all change it under load.

CONCEPT	DEFINITION	WARNING SIGN
Utilisation	Fraction of time the CPU is busy	Sustained > 80% — queueing begins
Saturation	Work waiting because the CPU is full	Run queue length exceeds core count
Context switching	Swapping between runnable threads	Thousands per second per core wastes real capacity
Throttling	Container CPU quota enforced	Latency spikes with utilisation apparently below 100%

21 Memory calculations

BUDGETING 32 GB

```
Total RAM             = 32 GB

Operating system      =  4 GB
Application heap      = 10 GB
PostgreSQL            =  8 GB
Redis cache           =  4 GB
                        ─────
Allocated             = 26 GB
Remaining headroom    =  6 GB
```

Six gigabytes of headroom is not slack — it absorbs connection buffers, request bodies held in flight, and the memory spike that accompanies a traffic surge. Under load, concurrency rises (§5), and every in-flight request holds memory until it completes.

REGION	HOLDS	GROWS WITH
Heap	Objects with dynamic lifetime	Concurrency and payload size
Stack	Call frames, one region per thread	Thread count
Buffers	Socket send/receive queues	Connection count
Page cache	Recently read file/disk pages	Working-set size — usually beneficial

MEMORY PRESSURE APPEARS AS LATENCY, NOT AS ERRORS

Before anything runs out of memory, the symptoms are **more frequent garbage collection** (in managed runtimes) and **page-cache eviction** (so reads that used to hit memory now hit disk). Both show up as rising p99 while utilisation graphs look calm. A soak test (§26) is the reliable way to expose this.

22 Database calculations

EXACT

$$\text{Query rate} = \text{RPS} \times \text{Queries per request}$$

Exact given a fixed query count per request. In practice the count varies by endpoint and by cache outcome — see §23.

AT CURRENT LOAD

```
100 RPS × 3 queries
  = 300 queries/second
```

AT 5× LOAD

```
500 RPS × 3 queries
  = 1,500 queries/second
```

API THROUGHPUT AND DATABASE THROUGHPUT ARE DIFFERENT NUMBERS

A modest 100 RPS at the API becomes **300 queries per second** at the database. Capacity planning that watches only the API rate will miss the tier that actually fails first. And if an endpoint has an N+1 query problem, the multiplier is not 3 but $3 + N$ — which is why load tests so often surface N+1 bugs that functional tests never noticed.

Connection pools — the most common hidden ceiling

```
POOL SIZING BY LITTLE'S LAW

Query rate      = 300 queries/second
Query duration  = 0.010 s

Connections in use = 300 × 0.010 = 3 connections

If the query slows to 0.100 s:
Connections in use = 300 × 0.100 = 30 connections
```

A pool of 20 connections is comfortable in the first case and **exhausted** in the second. Once exhausted, application threads block *waiting for a connection* — time that appears as application latency, with no database error to point at. This is one of the most misdiagnosed failure modes in web systems.

23 Cache hit ratio

EXACT

$$\text{Hit Ratio} = \text{Hits} \div \text{Total} \times 100$$

```
REDIS IN FRONT OF POCKET COACH READS

Total requests  = 1,000
Cache hits      =   900
Cache misses    =   100

Hit ratio = 900 / 1,000 × 100 = 90%
```

```
WHAT THAT DOES TO DATABASE LOAD

Without cache:  50 RPS × 3 queries              = 150 queries/s
With 90% hits:  50 RPS × 3 queries × (1 - 0.90) =  15 queries/s

Reduction factor = 10×
```

HIT RATIO IS A LOAD-DEPENDENT VARIABLE, NOT A CONSTANT

A load test that hammers **the same record** thousands of times will show a 99% hit ratio that your real, diverse traffic will never achieve — making the database look far healthier than it is. Conversely, a cold cache at test start understates it. **Parameterise your test data** so the spread of keys resembles production, and always report the hit ratio alongside the latency figures.

The arithmetic also explains cache-failure blast radius: if Redis becomes unavailable, database load jumps from 15 to 150 queries per second instantly — a **10× step change** with no warning. Testing that scenario deliberately is worthwhile.

24 Queueing: why latency explodes

Everything above assumes work is served as it arrives. When arrivals exceed service capacity, a queue forms — and queues behave non-intuitively.

ARRIVALS EXCEED SERVICE BY 25%

```
Arrival rate  = 100 requests/second
Service rate  =  80 requests/second

Backlog growth = 100 - 80 = 20 requests/second

After 60 seconds:
Queue length   = 20 × 60 = 1,200 requests
Wait for the last arrival = 1,200 / 80 = 15 seconds
```

A 25% OVERLOAD DOES NOT MAKE THINGS 25% SLOWER

It makes them **unboundedly** slower. The queue grows for as long as the overload persists, so response time grows without limit — from 200 ms to 15 seconds in one minute here. This is why systems appear to fail suddenly rather than gradually.

The degradation sequence

1 · QUEUE	Arrivals exceed service. Requests wait before processing begins.
2 · LATENCY	Queue wait is added to every response time. p99 moves first, then p95.
3 · CONCURRENCY	By Little's Law, higher *W* means higher *L* — more requests held in memory at once.
4 · EXHAUSTION	Threads, connections or memory run out. New work is refused or blocks.
5 · TIMEOUTS	Clients give up. 504s appear. Work already done is wasted.
6 · RETRIES	Clients retry, *adding* load to an overloaded system (§32). The failure accelerates.

UTILISATION AND LATENCY ARE NOT LINEARLY RELATED

Standard queueing results show waiting time scaling roughly with `1 / (1 - U)`, where *U* is utilisation. Going from 50% to 60% utilisation barely moves latency; going from 90% to 95% roughly *doubles* it. This is the mathematical reason capacity planning targets 70–80% rather than 95% — the last slice of utilisation is bought with disproportionate latency. The exact relationship depends on the arrival and service distributions, so treat this as the shape of the curve rather than a formula to compute with.

PART IV **Applying it**

25 Capacity, saturation and breaking point

Run the same scenario at increasing load and record throughput, latency and errors at each level. The shape of the resulting curve is the answer to “how many users can we handle?”

VUs	Throughput	p95	Errors	Interpretation
100	50 RPS	200 ms	0%	Healthy — linear region
200	98 RPS	250 ms	0%	Still linear — doubling load nearly doubled throughput
400	182 RPS	400 ms	0.1%	Knee — throughput gain falling behind load
600	222 RPS	900 ms	2%	Saturation — latency rising fast, throughput nearly flat
800	186 RPS	2.5 s	10%	Past breaking point — throughput *fell*

Throughput computed as $\lambda = N / (R + Z)$ with a constant think time $Z = 1.8$ s, so every row is internally consistent with Little's Law.

THE SIGNATURE OF A BROKEN SYSTEM: THROUGHPUT GOES DOWN

Between 600 and 800 VUs, load increased by a third while throughput **dropped from 222 to 186 RPS**. This is the defining symptom of saturation collapse — the system is spending its capacity on queueing, context switching, timeouts and retries rather than on useful work. Adding load past this point makes the system serve *fewer* customers, not more.

Term	Definition	In the table above
Capacity	Maximum load meeting your SLA	~400 VUs / 182 RPS if the SLA is p95 < 500 ms
Knee	Where latency starts rising disproportionately	Between 200 and 400 VUs
Saturation	A resource is fully consumed; queueing dominates	600 VUs
Breaking point	Throughput declines as load increases	Between 600 and 800 VUs
Bottleneck	The specific resource that saturates first	Not visible here — requires server-side metrics

THIS TABLE ALONE CANNOT NAME THE BOTTLENECK

Client-side numbers tell you *that* the system saturated and roughly *where*. They cannot tell you *why*. Naming the bottleneck requires server-side evidence gathered during the same run: CPU per tier, memory, connection-pool waits, database query time, disk I/O. Always collect both sides, or your report ends at “it got slow”.

26 The five test types compared

TYPE	QUESTION IT ANSWERS	TYPICAL SHAPE	WATCH FOR
Smoke	Does the test itself work?	1–2 VUs, 30–60 s	Auth working, IDs valid, no 4xx storm
Load	Do we meet the SLA at expected traffic?	Ramp to expected peak, hold 15–30 min	p95/p99 against threshold, error rate, steady throughput
Stress	Where does it break, and how?	Ramp well past peak until failure	The knee, the breaking point, whether failure is graceful
Spike	Can we survive a sudden surge and recover?	Baseline → 10–20× in seconds → baseline	Error burst, autoscaling lag, **recovery time**
Soak	Does it survive sustained normal traffic?	Moderate load, 4–24 hours	Latency drift, memory growth, connection leaks

SOAK TEST VOLUME — 8 HOURS AT 10 RPS

```
Requests = 10 × 3,600 × 8
         = 288,000 requests

A leak of just 1 KB per request would consume
288 MB over the run — invisible in a 5-minute test.
```

WHY SOAK TESTS FIND WHAT NOTHING ELSE CAN

Memory leaks, connection leaks, file-descriptor leaks, log-disk exhaustion and slow index degradation are all **functions of cumulative work**, not of instantaneous load. A short test at high load cannot find them; only time at moderate load can. Look for a *trend* in p95 across the hours — a flat line is the pass condition.

27 How tools turn configuration into traffic

Regardless of which tool you use, the same four responsibilities exist. The differences between tools are choices about *how* to discharge them.

SCHEDULE	Decide when each virtual user starts and when it repeats. Closed models start the next iteration when the previous finishes; open models start on a clock.
EXECUTE	Build the request, apply auth and variables, write it to a socket, read the response, run assertions.
MEASURE	Timestamp before and after, classify the outcome, attach tags (endpoint, status, scenario).
AGGREGATE	Fold samples into counters, rates and percentile structures — usually incrementally, because storing every sample is expensive.

The concurrency model is the main design axis

MODEL	HOW VUS MAP TO THE OS	TRADE-OFF
Thread-per-VU	One OS thread per virtual user	Simple and easy to reason about; memory and context-switching cost grow with VU count
Lightweight tasks	Many VUs multiplexed onto a few OS threads	Far more VUs per machine; requires non-blocking I/O throughout
Event loop	Single-threaded loop with async I/O	Very low overhead per connection; one slow callback blocks everything
Multi-process	Separate processes, often one per core	Uses all cores and isolates failures; results must be merged

PERCENTILE STORAGE IS AN ENGINEERING TRADE-OFF

Exact percentiles require every sample. At 10,000 RPS for an hour that is 36 million values per metric. Tools therefore use **reservoir sampling**, **histograms** or **t-digest** structures — each approximate in a different way. When two tools disagree slightly on p99, the storage strategy is often the reason, not the measurement.

28 The identical scenario in two tools

THE SCENARIO BOTH TOOLS MUST IMPLEMENT

```
500 virtual users
20 requests/second target
5-minute test
3 API requests per iteration
```

First: those numbers conflict

500 VUs and 20 RPS cannot both be free parameters. Little's Law fixes the relationship:

RECONCILING THE SPECIFICATION

$$N = \lambda \times (R + Z)$$

$$500 = 20 \times (R + Z)$$
$$R + Z = 25 \text{ seconds per iteration}$$

With 3 requests per iteration at $R = 0.2$ s each:
Work per iteration $= 0.6$ s
Think time needed $= 25 - 0.6 = 24.4$ s per iteration

$$\text{Total requests} = 20 \times 300 = 6{,}000$$
$$\text{Iterations} = 6{,}000 / 3 = 2{,}000$$

A SPECIFICATION WORTH CHALLENGING

Requiring 500 VUs to produce only 20 RPS means each simulated user acts once every 25 seconds. That may be realistic for a reading-heavy app — or it may mean whoever wrote the requirement conflated "users" with "requests". **Reconcile the numbers before running anything.** If you cannot make them agree, the specification is the defect.

What differs between implementations

ASPECT	WHY TOOLS DIFFER
VU representation	OS threads versus lightweight tasks — changes memory and scheduling cost per VU
Request scheduling	Whether the tool targets a VU count, an arrival rate, or both
Connection handling	Pool size, keep-alive defaults, HTTP/2 support
Runtime overhead	Managed runtimes pause for garbage collection; native ones do not
Metric aggregation	Exact samples versus histograms versus sketches
Result output	Writing every sample to disk costs I/O that a summary-only tool avoids

ON COMPARING MEASUREMENTS BETWEEN TOOLS

Two tools running "the same" test can report different latencies, and the difference is **not** a fixed per-request constant. It varies with concurrency, payload size, connection reuse, TLS, logging configuration and how much CPU headroom the generator has. Any claim of the form "tool X always adds *n* ms" should be rejected. **If the difference matters to your decision, measure it in your environment using the calibration method in §29** — and re-measure at the concurrency you actually intend to run.

29 Measuring your generator's overhead

Since generator overhead is variable, the only defensible figure is one you measure. The method is simple and worth doing once per environment.

STEP 1 Build a target of **known, fixed latency** — a trivial endpoint that sleeps for exactly *T* milliseconds and returns a small body. A timer, not computation, so the target uses almost no CPU.

STEP 2 Run your load tool against it at the **same concurrency** your real test will use.

STEP 3 Compute `ε = reported - T`. That is your instrument's overhead *at that concurrency*.

STEP 4 Repeat at several concurrency levels. Overhead typically **grows** with concurrency — the curve matters more than any single point.

EXACT

$$\varepsilon = T_{reported} - T_{true}$$

Valid only because T_{true} is known by construction. This is the one situation in load testing where you have ground truth — which is exactly why the calibration is worth building.

Judging whether a run is trustworthy

DISCARD THIS RESULT

```
Target reports  : p95 = 500 ms
Generator CPU   : 98%
Generator RAM   : 95%
Network         : saturated

The generator is itself queueing.
The target may never have received
the intended load. Any conclusion
about the target is unfounded.
```

TRUST THIS RESULT

```
Target reports  : p95 = 500 ms
Generator CPU   : 45%
Generator RAM   : 50%
Network         : 30%

Ample headroom. Requests were issued
on schedule. The measured latency is
dominated by the target, not by the
measuring device.
```

A PRACTICAL ACCEPTANCE RULE

Keep the generator below roughly **70–80% on every resource** — CPU, memory, and network. Record those figures in the test report alongside the latency numbers. A performance report that does not state the generator's own utilisation is incomplete, because the reader cannot judge whether the measurement is sound.

30 Network bandwidth

APPROXIMATION

$$\text{Bandwidth} \approx \text{RPS} \times (\text{Request size} + \text{Response size})$$

An approximation: it ignores TCP/IP and TLS framing overhead, headers, retransmissions and handshake traffic. Real utilisation typically runs 5–15% above this for small payloads, where header overhead is proportionally large.

POCKET COACH AT 50 RPS

```
Request size   =  2 KB
Response size  = 20 KB

Per second = 50 × (2 + 20)
           = 1,100 KB/s
           = 1.07 MB/s
           ≈ 9 Mbit/s
```

BITS VERSUS BYTES — A FACTOR OF EIGHT

Network links are rated in **bits** per second; payloads are measured in **bytes**. Multiply by 8 to convert. A "100 Mbps" link carries at most 12.5 MB/s, and considerably less after protocol overhead. Confusing the two produces capacity estimates that are wrong by 8× — in the dangerous direction.

WHEN BANDWIDTH BECOMES THE CEILING

```
Target throughput = 1,000 RPS
Response size     = 20 KB

Required = 1,000 × 22 KB = 22,000 KB/s
         ≈ 21.5 MB/s
         ≈ 172 Mbit/s

On a 100 Mbit/s link this is impossible.
The generator would cap at roughly 580 RPS —
and you would wrongly conclude the server
could not exceed that.
```

31 Timeouts

TIMEOUT	GOVERNS	IF SET TOO HIGH	IF SET TOO LOW
Connection	Establishing TCP/TLS	Threads block on dead hosts	Spurious failures under normal jitter
Read / socket	Waiting for response bytes	Resources held during outages	Slow-but-valid responses discarded
Client (total)	The whole request	Test appears to hang	Errors that the server would have answered
Server	How long the server will work	Workers occupied by hopeless requests	Legitimate long operations killed
Database	Query execution	Locks held, contention spreads	Heavy reports fail routinely

TIMEOUTS CONVERT LATENCY INTO ERRORS

Your measured error rate is partly a *configuration choice*. The same degraded system, tested with a 30-second client timeout, shows high latency and few errors; with a 2-second timeout it shows lower latency and many errors — because slow responses are discarded before they can be counted. **Always state the timeout alongside the error rate**, or the two numbers cannot be compared between runs.

THE TIMEOUT ORDERING RULE

Client timeout should exceed server timeout, which should exceed database timeout. When a client gives up first, the server keeps working on a request nobody will read — consuming capacity to produce nothing. Under load that wasted work compounds and accelerates collapse.

32 Retries and traffic amplification

Retries are a reliability feature that becomes an attack on your own system precisely when it is already struggling.

10% FAILURE RATE WITH AUTOMATIC RETRY

```
Original requests           1,000
10% fail → retried            100
10% of those fail → retried    10
and again                       1
                            _____
Total requests              1,111

Amplification: 11.1% more load than intended
```

EXACT

$$\text{Total} = N \div (1 - p)$$

N — original requests · **p** — failure probability. The sum of the geometric series of retry waves, assuming unlimited retries and a constant failure probability. Both assumptions are optimistic — real failure probability *rises* as the added load makes things worse.

THE SAME FORMULA AT 30% FAILURE

```
Total = 1,000 / (1 - 0.30)
      = 1,429 requests

Amplification: 43% additional load —
arriving exactly when the system is
least able to absorb it.
```

THE RETRY DEATH SPIRAL

Load rises → failures rise → retries add load → failures rise further → more retries. Each turn of the loop makes the next worse. This is why a system can fall from "slightly degraded" to "completely down" in seconds. Mitigations are **exponential backoff**, **jitter**, **retry budgets** and **circuit breakers** — all of which are worth load-testing explicitly, because they only ever activate under exactly these conditions.

33 Open vs closed workload models

This distinction determines what your test can and cannot discover. It is the single most consequential modelling choice available to you.

CLOSED MODEL — FIXED USERS

```
N virtual users loop forever.
A user cannot start request k+1
until request k has returned.

λ = N / (R + Z)

Server slows → λ falls automatically.
The test applies BACK-PRESSURE.
```

OPEN MODEL — FIXED ARRIVAL RATE

```
Requests start on a clock at
rate λ, regardless of whether
earlier ones have finished.

λ = configured constant

Server slows → concurrency grows.
The test applies NO back-pressure.
```

SAME DEGRADATION, TWO VERY DIFFERENT OUTCOMES

```
Baseline: R = 0.2 s, Z = 1.8 s, N = 100 VUs → λ = 50 RPS

Now the server degrades to R = 2.0 s:

CLOSED:  λ = 100 / (2.0 + 1.8) = 26.3 RPS
         Load DROPS by half. Queue stays bounded.
         Reported latency ≈ 2 s.

OPEN:    λ stays at 50 RPS by definition.
         In-flight = 50 × 2.0 = 100 concurrent requests
         (was 10). Queue grows. Latency keeps climbing.
```

WHY THIS MATTERS MORE THAN TOOL CHOICE

A closed-model test **protects the system from itself**. When things slow down, the simulated users politely wait, so the queue never explodes and the test reports a survivable-looking result. Real customers do not wait — they arrive on their own schedule, like an open model. **A closed-model test can report "pass" for a system that would collapse in production.**

COORDINATED OMISSION

The closed-model artefact above has a name: **coordinated omission**. Because the load generator waits for a slow response before issuing the next request, it silently stops sampling during exactly the periods when the system is worst. The measured latency distribution therefore *under-represents the tail* — sometimes by an order of magnitude. It is a property of the workload model, not of any particular tool, and every major tool offers an arrival-rate mode that avoids it. Use one for capacity work.

34 Thresholds and error budgets

TURNING AN SLO INTO A PASS/FAIL NUMBER

```
Requests in the test      = 100,000
Allowed failure rate      = 1%

Maximum allowed failures  = 100,000 × 0.01
                          = 1,000 failures

1,001 failures → the build fails.
```

THRESHOLD	EXAMPLE	WHAT IT PROTECTS
Latency percentile	`p95 < 500 ms`	The experience of almost every user
Tail latency	`p99 < 1 s`	The unlucky minority; often the loudest complainers
Error rate	`errors < 1%`	Correctness under load
Throughput floor	`RPS > 200`	Catches saturation collapse (§25)
Check success	`checks > 99%`	Responses are valid, not merely 200 OK

INCLUDE A THROUGHPUT FLOOR

Latency and error thresholds alone can be satisfied by a system that has quietly stopped doing work — serving very few requests, all of them quickly and successfully. A minimum-throughput threshold is what catches the breaking-point collapse where RPS *falls* as load rises.

SLA, SLO AND SLI

SLI is the measurement (observed p95). **SLO** is your internal target (p95 < 500 ms). **SLA** is the contractual promise with consequences, and is normally set looser than the SLO so you have room to react. Load-test thresholds should be pinned to the *SLO*, not the SLA — you want the build to fail before the customer notices.

35 Reading a real result

THE RUN TO INTERPRET

```
VUs             = 500
Requests        = 100,000
Throughput      = 333 RPS
Average         = 450 ms
p95             = 900 ms
p99             = 1.8 s
Failures        = 2,000
App server CPU  = 92%
App server RAM  = 78%
Database CPU    = 95%
```

First, derive what was not given

DERIVED FIGURES

```
Duration      = 100,000 / 333            = 300 s (5 minutes)
Error rate    = 2,000 / 100,000 × 100    = 2.0%
In-flight     = 333 × 0.450              = 150 concurrent requests
Implied think = 500/333 - 0.450          = 1.05 s per iteration
```

WHAT LOOKS ACCEPTABLE

- Throughput is substantial and steady
- p95 at 900 ms may meet a 1 s SLO
- RAM at 78% has headroom
- Ratio p99/p95 = 2.0 — a normal, not pathological, tail

WHAT IS ALARMING

- Database CPU 95% — effectively saturated, no headroom at all
- App CPU 92% — also near ceiling
- Error rate 2% = 2,000 real failures
- p99 is 4× the average — the tail is already stretching

WHERE THE BOTTLENECK MOST LIKELY IS

The database. It is the most saturated resource at 95%, and application CPU at 92% may partly reflect threads spinning while waiting on database work. Per §24, both tiers are deep in the region where a small load increase produces a large latency increase — which is consistent with the stretched p99.

Evidence still required before concluding

- **What are the 2,000 failures?** 500s mean defects; 504s mean saturation; 429s mean rate limiting worked. The status breakdown changes the conclusion entirely.
- **Were failures spread evenly or clustered?** A burst at one moment suggests a transient event; a rising rate suggests progressive saturation.
- **What was the load generator doing?** Without its CPU and network figures, we cannot rule out that we measured the generator (§29).
- **Connection-pool wait time.** This distinguishes "the database is slow" from "we cannot get a connection to the database".
- **Was this steady state?** Averages spanning the ramp understate the peak (§13).
- **Which endpoints failed?** A single bad endpoint is a different problem from uniform degradation.

CAN WE DECLARE THE SYSTEM READY FOR 500 USERS? NO.

Not because the numbers are bad, but because **there is no headroom left**. At 95% database CPU the system is at its ceiling with zero margin for traffic growth, a slow query, a failed cache, or a retry storm. The honest statement is: *"The system serves this load today at 2% errors, with no spare capacity. Any increase, or the loss of the cache, will cause failures."* Capacity means meeting the SLA **with margin** — see §36.

PART V **Reference**

36 Complete end-to-end worked example

Every calculation for Pocket Coach, from the manager's sentence to a capacity verdict. Each step feeds the next.

Step 1 — From population to throughput

```
GIVEN
Registered users            = 5,000
Peak concurrent users       =   500
Requests per user per minute =    6
```

```
DERIVE THE LOAD
Requests per minute = 500 × 6      = 3,000
Requests per second = 3,000 / 60   = 50 RPS

Time between one user's requests = 60 / 6 = 10 s
```

Step 2 — Think time and concurrency

```
LITTLE'S LAW, BOTH FORMS
Response time  R = 0.2 s
Think time     Z = 10 - 0.2 = 9.8 s

Requests in flight:  L = λ × R
                       = 50 × 0.2      = 10 requests

Check the user count: N = λ × (R + Z)
                        = 50 × 10.0    = 500 users  ✓
```

TWO ROUTES, ONE ANSWER

500 concurrent users generate 50 RPS and hold only **10 requests in flight**. Each user is in the system 2% of the time. Sizing for 500 simultaneous requests would over-provision by 50×.

Step 3 — Test configuration

WHAT TO TYPE INTO THE TOOL

```
Model            arrival rate (open) — avoids coordinated omission
Rate             50 requests/second
Duration         30 minutes steady state
Ramp-up          5 minutes  (50 / 300 = 0.167 RPS added per second)
Scenario         4-request journey
Think time       distributed around 9.8 s

Total requests = 50 × 1,800 = 90,000 (steady state only)
```

Step 4 — Traffic generated

BANDWIDTH

```
Request 2 KB, response 20 KB

50 × (2 + 20) = 1,100 KB/s
              ≈ 1.07 MB/s
              ≈ 9 Mbit/s          (approximation — see §30)
```

Step 5 — What the tiers receive

DATABASE AND CACHE

```
Queries per request  = 3
Query rate           = 50 × 3            = 150 queries/s

With 90% cache hit ratio:
Reaching PostgreSQL  = 150 × (1 - 0.90)  = 15 queries/s

Connections needed (query = 10 ms):
                     = 150 × 0.010       = 1.5 connections
If the query degrades to 100 ms:
                     = 150 × 0.100       = 15 connections
```

CPU (SERVICE DEMAND D = 0.040 S, MEASURED)

```
Cores busy   = 50 × 0.040        = 2.0 of 16 cores
Utilisation  = 2.0 / 16 × 100    = 12.5%

CPU ceiling  = 16 / 0.040        = 400 RPS
Safe at 70%  = (16 × 0.70)/0.040 = 280 RPS
```

```
MEMORY
32 GB total -  OS 4  -  app 10  -  DB 8  -  Redis 4
            = 6 GB headroom
```

Step 6 — Thresholds

```
PASS/FAIL CRITERIA
p95 < 500 ms
p99 < 1,000 ms
error rate < 1%   →  90,000 × 0.01 = 900 failures allowed
throughput > 45 RPS  (catches saturation collapse)
```

Step 7 — Capacity verdict with headroom

```
DESIGN FOR GROWTH, NOT FOR TODAY
Today's peak                =  50 RPS
Safety factor               =  2×  (growth, spikes, cache loss)
Design target               = 100 RPS

CPU needed at 70% utilisation:
  cores = (100 × 0.040) / 0.70   = 5.7 cores

Available: 16 cores → comfortable.
CPU is NOT the binding constraint at this load.
```

```
THE VERDICT
```

Pocket Coach's application tier has substantial CPU margin — 12.5% utilisation at peak, with a theoretical ceiling around 400 RPS. **The risks are elsewhere:** the database at 15 queries/second is fine *only while the cache holds*. Lose Redis and database load steps to 150 queries/second instantly (§23). The connection pool must be sized for degraded query times, not healthy ones (§22). Those two scenarios — cache failure and slow queries — are what this system should be tested against, not raw user count.

```
WHICH OF THESE NUMBERS ARE REAL?
```

Only **D = 0.040 s** and **R = 0.2 s** would come from measurement; everything else is derived from them by arithmetic. The whole model is therefore only as good as those two measurements. **Measure them first, on the real system, before trusting any conclusion built on top.**

37 Mathematical accuracy rules

Every quantity in a performance report belongs to one of four categories. Labelling them is what separates an engineering document from a guess.

CATEGORY	MEANING	EXAMPLE FROM THIS DOCUMENT
Exact	Follows by definition or proven theorem	`RPS = requests / seconds`; Little's Law in steady state
Approximation	Ignores real effects for tractability	Bandwidth ≈ RPS × payload (ignores framing)
Assumption	A chosen input that could be otherwise	3 queries per request; 90% cache hit ratio
Measurement	Observed in a specific environment	Service demand *D*; response time *R*; generator overhead ε

CLAIMS TO REFUSE

- **"Tool X adds *n* ms to every request."** Generator overhead varies with concurrency, payload, TLS, connection reuse, logging and CPU headroom. It must be measured per environment (§29).
- **"Concurrency = RPS × response time" as a universal law.** True as Little's Law only in *steady state*. During a spike, or while a queue grows without bound, it does not describe the system.
- **"We support N users" without stating think time.** §15 shows the same 100 VUs spanning 10–500 RPS. A VU count alone is not a load specification.
- **"Average response time is X."** §11 shows an average that describes none of the 100 users measured. Report percentiles.
- **Averaged percentiles.** p95 values cannot be averaged across machines or time buckets. Recompute from samples.
- **"16 cores at 75% = 12 cores of capacity" taken literally.** Ignores serial sections, hyper-threading and the non-linear latency cost of high utilisation.

THE INTERNAL-CONSISTENCY CHECK

Before publishing any table of load figures, verify that each row satisfies `N = λ × (R + Z)` with a *consistent* think time. A row implying a negative think time is physically impossible and reveals invented numbers. This check takes seconds and catches a surprising number of published benchmarks.

38 Formula cheat sheet

QUANTITY	FORMULA	STATUS
Throughput	`RPS = Total Requests / Total Seconds`	Exact
Total requests (open)	`Requests = Arrival Rate × Duration`	Exact
Total requests (closed)	`Requests = Users × Iterations × Requests per iteration`	Exact
Total iterations	`Iterations = Users × Iterations per user`	Exact
Little's Law	`L = λ × W`	Exact (steady state)
Concurrency of requests	`In-flight = RPS × Response Time`	Exact (steady state)
Closed-model users	`N = λ × (R + Z)`	Exact (steady state)
Throughput from users	`λ = N / (R + Z)`	Exact (steady state)
Utilization Law	`U = X × D`	Exact
CPU ceiling	`Max RPS = Cores / D`	Approx (perfect parallelism)
Safe throughput	`Safe RPS = (Cores × 0.70) / D`	Approx (target utilisation)
Error rate	`Errors % = Failed / Total × 100`	Exact
Success rate	`Success % = Successful / Total × 100`	Exact
Allowed failures	`Max Failures = Total × Allowed Rate`	Exact
Database query rate	`QPS = RPS × Queries per Request`	Exact (fixed ratio)
Queries after cache	`QPS = RPS × Q × (1 - Hit Ratio)`	Exact (fixed ratio)
Cache hit ratio	`Hit % = Hits / Total × 100`	Exact
Connections in use	`Connections = QPS × Query Duration`	Exact (Little's Law)
Bandwidth	`Bytes/s ≈ RPS × (Req Size + Resp Size)`	Approx (ignores framing)
Bits from bytes	`bits = bytes × 8`	Exact
Queue growth	`Backlog/s = Arrival Rate - Service Rate`	Exact
Queue wait	`Wait = Queue Length / Service Rate`	Exact (FIFO)
Retry amplification	`Total = N / (1 - p)`	Approx (constant p, unlimited retries)
Ramp rate	`Users/s = VUs / Ramp Seconds`	Exact (linear ramp)
Generator overhead	`ε = Reported - Known True Latency`	Exact (known target)
Generators needed	`Machines = Target RPS / (Per-machine × 0.70)`	Approx (headroom)
Soak volume	`Requests = RPS × 3600 × Hours`	Exact

39 Terminology glossary

TERM	DEFINITION, ANALOGY AND EXAMPLE
Virtual User (VU)	A simulated worker performing one action at a time, then repeating. *Analogy:* one shopper walking the aisles. *Example:* 100 VUs at R = 0.2 s, Z = 1.8 s produce 50 RPS.
Concurrent user	A real person with a live session at a given instant. *Analogy:* people currently inside the shop. *Example:* 500 concurrent users generating only 10 simultaneous requests.
Active user	Someone who used the system within a period, typically a day. Larger than concurrent, smaller than registered.
Registered user	A row in the users table. Carries no load information whatsoever.
Throughput	Work completed per unit time. *Analogy:* customers served per hour. *Example:* 12,000 requests / 600 s = 20 RPS.
RPS	Requests per second — HTTP calls completed each second.
TPS	Transactions per second — business actions. One transaction may be several requests: 20 TPS × 4 = 80 RPS.
Latency	Strictly, network round-trip time. Loosely used for response time — always disambiguate. *Example:* 40 ms RTT inside a 200 ms response.
Response time	Client-observed elapsed time from first byte sent to last byte received. Includes network, queueing and server work.
TTFB	Time to first byte — response time minus the body transfer.
p50 / median	Half of requests were faster. *Analogy:* the middle person in a queue by wait time.
p90 / p95 / p99	90 / 95 / 99 per cent of requests were faster. *Example:* p99 = 3.1 s means 1 in 100 users waited over 3 seconds.
Error rate	Failed ÷ total × 100. *Example:* 500 / 10,000 = 5%.
Arrival rate	Rate at which new requests or users enter, independent of completions. *Analogy:* people arriving at a door, whether or not there is room.
Iteration	One complete pass of a VU through the scenario. *Analogy:* one lap of the shop.
Think time	Simulated pause between actions. *Example:* 9.8 s between requests for a user acting 6 times per minute.
Ramp-up	Gradual increase to target load. *Example:* 100 VUs over 300 s = 0.333 VUs/second.
Ramp-down	Gradual decrease at the end, allowing in-flight work to finish cleanly.
Load test	Expected traffic held steady; verifies the SLA is met.
Stress test	Load increased past expectation to find the breaking point.
Spike test	Sudden multiplication of load; tests survival and recovery.
Soak / endurance test	Moderate load for hours. *Example:* 10 RPS for 8 hours = 288,000 requests. Finds leaks.
Smoke test	Minimal load confirming the test itself is correctly configured.

TERM	DEFINITION, ANALOGY AND EXAMPLE
Capacity	Maximum load still meeting the SLA, *with headroom*. Not the same as maximum achievable throughput.
Bottleneck	The resource that saturates first. *Analogy:* the narrowest point of a funnel — widening anything else changes nothing.
Saturation	A resource fully consumed, so work must queue.
Queue	Work waiting for a busy resource. Grows without bound while arrivals exceed service.
Connection pool	Reusable set of database connections. *Example:* 300 QPS × 10 ms = 3 in use; at 100 ms, 30 in use — exhausting a pool of 20.
Cache hit / miss	Whether data was served from cache. *Example:* 900 / 1,000 = 90% hit ratio, cutting database load tenfold.
CPU utilisation	Fraction of time the processor is busy. Above ~80%, queueing makes latency rise sharply.
Service demand (D)	CPU-seconds consumed per request. *Example:* D = 0.04 s means 16 cores can theoretically serve 400 RPS.
Memory utilisation	Fraction of RAM in use. Pressure appears as garbage collection and cache eviction before it appears as an error.
Network throughput	Bytes per second across the link. Rated in bits — multiply bytes by 8.
Timeout	How long a party waits before abandoning a request. Converts latency into errors, so it shapes your error rate.
Retry	Re-sending a failed request. *Example:* at 10% failure, total load = N / (1 − 0.1) = 1.11 N.
Load generator	The machine producing the load. Subject to the same limits as the system under test.
Distributed testing	Several coordinated generators. *Example:* 10,000 RPS at 1,400 usable per machine = 8 machines.
Threshold	A pass/fail rule evaluated against a metric, e.g. `p95 < 500 ms`.
SLI	Service Level Indicator — the measurement itself.
SLO	Service Level Objective — your internal target for that indicator.
SLA	Service Level Agreement — the contractual promise, with consequences. Normally looser than the SLO.
Baseline	A recorded reference result used to detect change over time.
Regression	Measurable degradation against the baseline — the main reason to run load tests continuously.
Closed workload	Fixed user population; a user waits for a response before continuing. Applies back-pressure and hides tail latency.
Open workload	Requests arrive at a set rate regardless of completions. Models real customers; exposes queue growth.
Coordinated omission	Under-sampling exactly when the system is slowest, because the generator waits for responses. Under-reports tail latency; avoided with an arrival-rate model.
Headroom	Deliberate unused capacity absorbing growth, spikes and failure. Capacity without headroom is not capacity.

– The answer to the central question

“I configure a load test in a tool — what exactly happens from that moment until the final graph and numbers appear on my screen?”

YOU	Convert a population (“500 users”) into quantities a machine can generate: rate, duration, think time, thresholds. **Most bad load tests fail here.**
TOOL	Turns that configuration into a schedule and allocates workers. The workload model chosen here — open or closed — determines what the test is capable of discovering.
MACHINE	Spends real CPU on TLS, serialisation and metrics; consumes sockets and file descriptors. Any shortage here is silently attributed to your server.
NETWORK	Carries bytes, adding propagation delay and a bandwidth ceiling that can cap throughput before the server is touched.
SERVER	Queues, then processes. Queue time is invisible to server-side metrics but fully visible to your users.
DATA TIER	Cache decides how much reaches the database. The connection pool, sized by Little's Law, becomes the ceiling as queries slow.
RETURN	The tool stops its clock. The figure it records spans everything above — not just your application.
METRICS	Counts and rates are exact; percentiles are often approximate by construction, and can never be averaged.
YOU	Decide. Every distortion in the chain is now inside your number — which is why the generator's own utilisation belongs in the report beside the result.

IF YOU REMEMBER FOUR THINGS

1. **Users are not requests.** 500 concurrent users produced 50 RPS and 10 in-flight requests. State think time or state throughput — a VU count alone means nothing.
2. **The average describes nobody.** Report p95 and p99, and never average percentiles together.
3. **Measure the measuring device.** A saturated load generator invalidates the result; publish its utilisation alongside the numbers.
4. **Capacity includes headroom.** A system at 95% CPU with a 2% error rate is not “at capacity” — it is already past it.